PROGRAMMING IN JAVA

ASSIGNMENT 2 – CO2

STRINGS AND COLLECTIONS

Completed Assignment Report

Student details	Information
Name	S.Charmi
Register Number	192324034
Course Code	CSA0915
Subject	Programming in Java
GitHub Repository	https://github.com/Charmi-saravanan/CSA0915-JAVA-
Java Version	Java 17 or later
Submission Date	13-11-2026

Note on experimental evidence: this report supplies reproducible Java implementations, test designs, expected logical outcomes and complexity analysis. Hardware-dependent execution times and memory readings are deliberately not invented. Run the included benchmark plans on the submission computer and enter the measured averages in the clearly marked evidence tables before submission.

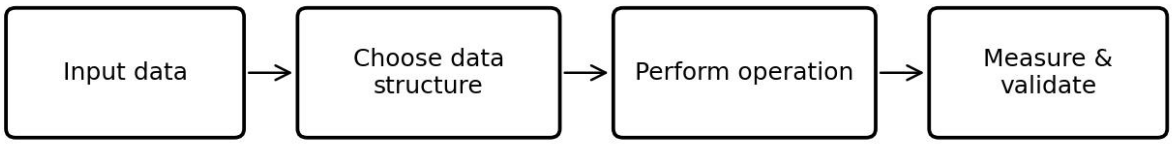
Table of Contents

- Introduction and execution methodology
- Q1 – String, StringBuilder and StringBuffer
- Q2 – ArrayList, LinkedList and ArrayDeque
- Q3 – hashCode–equals contract
- Q4 – Pharmacy inventory maps
- Q5 – Generic leaderboard and comparator correctness
- Q6 – Priority queue triage and fairness
- Q7 – Diagnose, redesign and validate
- Overall conclusion
- Appendix – GitHub structure and execution checklist

Introduction and Execution Methodology

This assignment investigates how Java strings and collection types behave under different workloads. The central engineering principle is that a data structure should be selected from the operations and ordering guarantees required by the application, not merely from familiarity or a single timing result.

For a fair experiment, use the same machine, Java version, input data, warm-up procedure and number of repetitions for every alternative. Measure with System.nanoTime(). Run each workload several times, discard initial warm-up runs where JIT compilation may distort results, and report the median or arithmetic mean together with the test size. Use System.gc() only as a best-effort hint; it does not provide a reliable exact memory measurement.



Recommended procedure: prepare deterministic input → execute a warm-up → run at least 5–10 measured repetitions → calculate average time → verify the result → record machine and JVM details.

Measure	Recommended method	Limitation
Elapsed time	System.nanoTime() around only the operation	JIT, GC and OS scheduling introduce variation
Approximate heap change	Runtime.totalMemory() - Runtime.freeMemory() before/after	Noisy; includes unrelated allocations
Object count	Analytical estimate or profiler/JFR	Do not claim exact counts from source inspection
Correctness	Assertions, expected order and	A fast incorrect result is not

	set/map sizes	acceptable

Q1 – Building Text Under Load: String, StringBuilder or StringBuffer? (6 Marks)

1.1 Prediction and design

String is immutable. Each concatenation in a loop creates a new result String and copies the previous content plus the new field. StringBuilder is mutable and appends into a growable character buffer, so it avoids repeatedly copying the complete accumulated ticket. StringBuffer has a similar mutable-buffer design but synchronizes its methods, adding synchronization overhead.

Workload	Description	Purpose
A – Small	1 ticket with 6 fields; repeat 100 times	Shows whether overhead is noticeable for tiny tickets
B – Large	10,000 tickets, 50 fields per ticket	Amplifies repeated concatenation and allocation
C – Long ticket	1,000 tickets, 500 fields per ticket	Tests growth when each ticket contains many fields

Approach	Prediction as fields grow	Intermediate allocation
String + in loop	Slowest for large tickets due to repeated copying	Many temporary String results
StringBuilder	Usually fastest for single-threaded ticket construction	One builder buffer plus final String
StringBuffer	Correct and thread-safe for shared individual operations; usually slightly slower than StringBuilder	Builder buffer plus synchronization overhead

1.2 Java implementation

```
import java.util.*;
public class TicketBenchmark {
    static String[] fields(int n) {
        String[] a = new String[n];
        for (int i = 0; i < n; i++) a[i] = "FIELD" + i + "=VALUE" + i;
        return a;
    }
    static String usingString(String[] a) {
        String s = "";
        for (String x : a) s += x + "\\n";
        return s;
    }
    static String usingBuilder(String[] a) {
        StringBuilder b = new StringBuilder();
        for (String x : a) b.append(x).append('\\n');
        return b.toString();
    }
    static String usingBuffer(String[] a) {
        StringBuffer b = new StringBuffer();
        for (String x : a) b.append(x).append('\\n');
```

```
        return b.toString();
    }
    static long measure(Runnable task) {
        long start = System.nanoTime(); task.run();
        return System.nanoTime() - start;
    }
    public static void main(String[] args) {
        int tickets = 10000, fieldCount = 50, repetitions = 7;
        String[][] data = new String[tickets][];
        for (int i=0;i<tickets;i++) data[i] = fields(fieldCount);
        for (int r=0;r<2;r++) { // warm-up
            for (String[] a:data) { usingString(a); usingBuilder(a); usingBuffer(a); }
        }
        long s=0,b=0,f=0;
        for (int r=0;r<repetitions;r++) {
            s += measure(() -> { for (String[] a:data) usingString(a); });
            b += measure(() -> { for (String[] a:data) usingBuilder(a); });
            f += measure(() -> { for (String[] a:data) usingBuffer(a); });
        }
        System.out.printf("String avg: %.3f ms%n",s/repetitions/1e6);
        System.out.printf("StringBuilder avg: %.3f ms%n",b/repetitions/1e6);
        System.out.printf("StringBuffer avg: %.3f ms%n",f/repetitions/1e6);
    }
}
```

1.3 Analysis of results and design decision

For a ticket with n fields whose total appended character count grows approximately in proportion to n, repeated concatenation may copy prefixes of lengths 1, 2, …, n. The total copied characters are proportional to n^2 in the simplified equal-sized-field model. StringBuilder appends amortized O(1) per character and copies the final content once in toString(), giving O(L) time in the total output length L (often described as O(n) when field sizes are bounded).

For each independent ticket, use StringBuilder. It is local to the request/thread and does not need synchronization. For a shared booking log, do not assume StringBuffer alone solves the complete logging problem: synchronizing each append prevents interleaving at method level, but a complete log record should be appended atomically. Prefer a dedicated logging framework or a synchronized/locked append operation around the complete record. StringBuffer is acceptable for a simple shared mutable buffer; StringBuilder is not safe for unsynchronized concurrent mutation.

Alternative: String.join(delimiter, fields) or StringJoiner builds a joined representation with linear work in total characters and is concise when fields are already available as a collection/array. String.format is convenient for formatted output but generally has greater formatting overhead and is not the best default for high-volume concatenation.

Method	**Time complexity (total output length L)**	**Extra space**
Repeated + in loop	$O(L^2)$ in the repeated-prefix copying model	Many intermediate strings; cumulative allocation can be quadratic
StringBuilder	O(L) amortized	O(L) growable buffer and final String
StringBuffer	O(L) amortized, plus synchronization cost	O(L) buffer and final String
String.join/StringJoiner	O(L)	O(L) output/buffer; concise API

1.4 Evidence table to complete after execution

Workload	String avg (ms)	Builder avg (ms)	Buffer avg (ms)	Observed winner
A: 100 × 6 fields	Enter measured value	Enter measured value	Enter measured value	Enter result
B: 10,000 × 50 fields	Enter measured value	Enter measured value	Enter measured value	Enter result
C: 1,000 × 500 fields	Enter measured value	Enter measured value	Enter measured value	Enter result

Conclusion: the difference can be negligible for a tiny ticket, but the repeated allocation and copying cost becomes important under large workloads. Use StringBuilder for per-ticket construction and a properly coordinated logging mechanism for shared records.

Q2 – Music Playlist Editor: ArrayList, LinkedList and ArrayDeque

2.1 Experimental plan and predictions

Use playlist sizes of 1,000, 10,000 and 100,000 songs. Prepopulate equivalent collections and time each operation independently. For middle operations, use the same index (for example, size/2). For LinkedList, use a ListIterator positioned at the target if the experiment is intended to measure insertion after locating the position separately; also report the cost of locating the position because normal indexed access is O(n).

Operation	ArrayList	LinkedList	Reason
Append at end	Amortized O(1)	O(1) with tail pointer	Both efficient
Insert/remove front	O(n)	O(1) once node/iterator is at position; locating index is O(n)	ArrayList shifts elements
Insert/remove middle by index	O(n)	O(n) to reach index; O(1) link change	LinkedList traversal dominates
Get k-th element	O(1)	O(n)	ArrayList supports direct indexing
Full iteration	O(n), cache-friendly	O(n), pointer traversal	ArrayList often has better locality

2.2 Java comparison program

```
import java.util.*;
public class PlaylistComparison {
    static long time(Runnable r) {
        long t=System.nanoTime(); r.run(); return System.nanoTime()-t;
    }
    public static void main(String[] args) {
        for (int n : new int[]{1000,10000,100000}) {
            List<Integer> a=new ArrayList<>(), l=new LinkedList<>();
            for(int i=0;i<n;i++){a.add(i);l.add(i);}
            int reps=1000, k=n/2;
            long ta=time(()->{for(int i=0;i<reps;i++) a.add(999999);});
            long tl=time(()->{for(int i=0;i<reps;i++) l.add(999999);});
            List<Integer> ac=new ArrayList<>(a), lc=new LinkedList<>(l);
            long af=time(()->{for(int i=0;i<reps;i++) ac.add(0,-i);});
```

```
            long lf=time(()->{for(int i=0;i<reps;i++) lc.add(0,-i);});
            long ag=time(()->{long x=0;for(int i=0;i<reps;i++)x+=a.get(k);});
            long lg=time(()->{long x=0;for(int i=0;i<reps;i++)x+=l.get(k);});
            long ai=time(()->{long x=0;for(int v:a)x+=v;});
            long li=time(()->{long x=0;for(int v:l)x+=v;});
            System.out.printf("n=%d append A/L=%d/%d front A/L=%d/%d get A/L=%d/%d iterate A/L=%d/%d%n",
                n,ta,tl,af,lf,ag,lg,ai,li);
        }
    }
}
```

The sample program is a starting benchmark, not a complete statistically rigorous harness. Repeat the whole measurement several times, include middle insertion/removal in separate timed blocks, and avoid reusing a list that has grown differently between alternatives. Use a fresh copy for each run.

2.3 Engineering recommendation and changed condition

ArrayList is a reasonable default for a playlist when users frequently jump to a song by index, iterate through the playlist, append occasionally, and perform relatively few insertions/removals near the front or middle. Its contiguous backing array provides fast random access and generally good cache locality.

Stop recommending ArrayList when the dominant workload performs frequent insertions/removals near the beginning or in the middle and random indexed access is rare. However, LinkedList is not automatically faster for middle edits if the application repeatedly locates an index; node traversal is still O(n), and each element requires a separate node object.

For the new Up Next queue, where songs are frequently added at both ends and removed from the front, ArrayDeque is the strongest fit. addFirst/addLast and removeFirst/removeLast are amortized O(1). It is more memory-efficient than LinkedList in typical queue use and avoids shifting array elements. It is not a List and does not provide efficient arbitrary indexed access.

Test size	Operation	ArrayList measured	LinkedList measured	Notes
1,000	Append/front/middle/get/iterate	Fill after run	Fill after run	Use same repetitions
10,000	Append/front/middle/get/iterate	Fill after run	Fill after run	Use same repetitions
100,000	Append/front/middle/get/iterate	Fill after run	Fill after run	Use same repetitions

Q3 – Engineer the hashCode–equals Contract (6 Marks)

3.1 Explanation

The equals() method defines logical identity: in this problem, two Student objects represent the same student when their registration numbers match. The Java contract requires that equal objects have equal hash codes. The reverse is not required: unequal objects may share a hash code.

HashSet uses hashCode() to select a bucket and then uses equals() to check candidate elements in that bucket. If equals() is overridden but hashCode() is inherited from Object, two separate Student

instances with the same regNo can have different identity-based hash values. They may be placed in different buckets, so the set may never compare them with equals(), and duplicates can remain.

3.2 Correct Student class and complete-code implementation

```
import java.util.HashSet;
import java.util.Objects;
import java.util.Set;


class Student {
    private final String regNo;
    private final String name;
    Student(String regNo, String name) {
        this.regNo = Objects.requireNonNull(regNo);
        this.name = name;
    }
    @Override public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Student)) return false;
        Student other = (Student)o;
        return regNo.equals(other.regNo);
    }
    @Override public int hashCode() {
        return Objects.hash(regNo);
    }
    @Override public String toString() { return regNo + " - " + name; }
}
public class StudentSetTest {
    public static void main(String[] args) {
        Set<Student> students = new HashSet<>();
        students.add(new Student("A101","Anu"));
        students.add(new Student("A102","Bala"));
        students.add(new Student("A101","Anu Kumar"));
        System.out.println("Unique students: " + students.size());
    }
}
```

Expected output after the correction:

```
Unique students: 2
```

Before implementing hashCode(), the exact set size is not guaranteed to be 2. It will commonly be 3 because the two distinct objects can have different identity hash codes. Since hash values can theoretically collide, the important explanation is the broken contract, not a promise that every run must print 3.

3.3 Three deliberate tests

Test	Input pattern	Prediction with correct contract
1 – Exact duplicate	A101/Anu scanned twice	Set size remains 1 for those two scans
2 – Same name, different ID	A101/Anu and A102/Anu	Both remain; registration number defines identity
3 – Same ID, changed name	A101/Anu and A101/Anu Kumar	Only one remains because regNo is identity

3.4 Mutation risk and alternatives

The supplied completed class makes regNo final, preventing it from changing after insertion. If a mutable field used by hashCode() changes after an object is inserted into a HashSet, the object remains stored in the bucket selected by its old hash. A later contains() or remove() call calculates the new hash and may search a different bucket. The object can become difficult to find or remove. Therefore, fields used in equality and hashing should be immutable while an object is stored in a hash-based collection.

Structure	De-duplication method	Average/per-operation cost	Ordering
HashSet<Student>	Override equals/hashCode using regNo	O(1) average add/contains	No guaranteed order
TreeSet<Student>	Comparator compares regNo; compare==0 means duplicate	O(log n)	Sorted by regNo
HashMap<String,Student>	putIfAbsent(regNo, student)	O(1) average	No guaranteed order

```
TreeSet<Student> unique = new TreeSet<>(
    Comparator.comparing(s -> s.getRegNo()));
// Or:
Map<String, Student> byRegNo = new HashMap<>();
byRegNo.putIfAbsent(student.getRegNo(), student);
```

Space: each structure stores O(n) entries for n unique students. HashSet/HashMap also maintain a backing hash table; TreeSet maintains tree nodes. HashMap is especially useful when the registration number is already the natural lookup key.

Q4 – Choosing a Map for a Pharmacy Inventory (6 Marks)

4.1 Map selection

The inventory has three distinct requirements: rapid lookup by medicine code, a report sorted by code, and a report in insertion order. No single ordinary Map implementation provides all three views with the same performance. HashMap is appropriate for average O(1) lookup, TreeMap maintains key order with O(log n) lookup/update, and LinkedHashMap preserves insertion order.

4.2 Java implementation

```
import java.util.*;
class Medicine {
    final String code, name;
    int quantity;
    Medicine(String c,String n,int q){code=c;name=n;quantity=q;}
    public String toString(){return code+" "+name+" qty="+quantity;}
}
public class PharmacyInventory {
    public static void main(String[] args) {
        Map<String,Medicine> fast = new HashMap<>();
        Map<String,Medicine> sorted = new TreeMap<>();
        Map<String,Medicine> insertion = new LinkedHashMap<>();
        Medicine[] data = {
            new Medicine("M103","Paracetamol",120),
            new Medicine("M101","Amoxicillin",60),
            new Medicine("M105","Cetirizine",45),
```

```
            new Medicine("M102","ORS",80),
            new Medicine("M104","Metformin",90)
        };
        for(Medicine m:data){
            fast.put(m.code,m); sorted.put(m.code,m); insertion.put(m.code,m);
        }
        System.out.println("Lookup M102: "+fast.get("M102"));
        System.out.println("Code order:");
        sorted.forEach((k,v)->System.out.println(v));
        System.out.println("Insertion order:");
        insertion.forEach((k,v)->System.out.println(v));
    }
}
```

Expected report order: code order is M101, M102, M103, M104, M105. Insertion order is M103, M101, M105, M102, M104. HashMap does not promise either order.

4.3 Benchmark plan and deployment criterion

Test at least 1,000, 10,000 and 100,000 medicines. Generate unique codes deterministically, perform 100,000 lookups using a pre-generated list of keys, and separately time the sorted report and insertion-order report. Do not include data creation in the lookup timer. Verify the exact output order.

A reasonable initial service-level criterion for an in-memory inventory lookup is a median below 1 millisecond per lookup on the target server; the actual hospital requirement should be agreed with stakeholders and tested on deployment hardware. For 100,000 total lookups, report total time and calculate average time per lookup.

Map	Lookup	Insert/update	Code-order report	Insertion-order report	Best use
HashMap	O(1) average	O(1) average	O(n log n) if keys are sorted separately	No guarantee	Fast lookup
TreeMap	O(log n)	O(log n)	O(n) traversal	No guarantee	Sorted-key reporting
LinkedHashMap	O(1) average	O(1) average	O(n log n) if separately sorted	O(n) traversal	Insertion-order reporting

A practical design can maintain a HashMap for fast lookup and create sorted/insertion-order views when reports are requested, or use separate TreeMap and LinkedHashMap views if the extra memory and update coordination are acceptable. If only one map is allowed and sorted reports are frequent, TreeMap is a defensible choice. If lookups dominate, HashMap plus report sorting is usually preferable.

Hashtable is generally not preferred in new code because it is a legacy synchronized Map implementation, disallows null keys and values, and offers coarse method-level synchronization. Modern applications normally use HashMap or concurrent collections such as ConcurrentHashMap when concurrency is required.

4.4 Sorted-array alternative

Keep an array/list sorted by medicine code and use Arrays.binarySearch(). Lookup is O(log n); insertion is O(n) because elements may need shifting; producing the code-order report is O(n) because the array is already sorted. The array uses O(n) storage and has good locality, but resizing and insertion

are expensive. HashMap provides O(1) average lookup but no order; TreeMap provides O(log n) updates and naturally sorted traversal.

Design	Lookup	Insertion	Sorted report	Space
Sorted array	O(log n)	O(n)	O(n)	O(n)
HashMap	O(1) average	O(1) average	O(n log n) to sort keys	O(n)
TreeMap	O(log n)	O(log n)	O(n)	O(n)

Medicine count	100,000 lookups time	Sorted report time	Insertion report time	Order verified
1,000	Enter measured value	Enter measured value	Enter measured value	Yes/No
10,000	Enter measured value	Enter measured value	Enter measured value	Yes/No
100,000	Enter measured value	Enter measured value	Enter measured value	Yes/No

Q5 – Derive a Generic Ranking Component from a Principle (7 Marks)

5.1 Design choice

A ranking is correct when the comparison is consistent and transitive. For a league table, teams are ordered by points descending, goal difference descending, and name ascending as a deterministic final tie-breaker. Comparator composition expresses this rule clearly.

Comparable<Team> is suitable when one natural ordering is universal for Team. Separate Comparator objects are better when the same type may be ranked in different ways, such as points, goal difference, alphabetical name, or fair-play score. The generic Leaderboard accepts Comparator<? super T>, allowing a comparator of T or one of its supertypes.

5.2 Complete generic code

```
import java.util.*;
class Team {
    final String name; final int points; final int goalDiff;
    Team(String n,int p,int g){name=n;points=p;goalDiff=g;}
    public String toString(){return name+"("+points+","+goalDiff+")";}
}
class Leaderboard<T> {
    private final List<T> items = new ArrayList<>();
    private final Comparator<? super T> order;
    Leaderboard(Comparator<? super T> order){this.order=order;}
    void add(T item){items.add(item);}
    void addAll(List<? extends T> values){items.addAll(values);}
    List<T> ranking(){
        List<T> copy=new ArrayList<>(items);
        copy.sort(order);
        return copy;
    }
}
public class LeaderboardDemo {
    public static void main(String[] args) {
        Comparator<Team> order =
```

```
            Comparator.comparingInt((Team t)->t.points).reversed()
            .thenComparing(Comparator.comparingInt(
                (Team t)->t.goalDiff).reversed())
            .thenComparing(t->t.name);
        Leaderboard<Team> board=new Leaderboard<>(order);
        board.addAll(Arrays.asList(
            new Team("Falcons",18,12),
            new Team("Comets",18,9),
            new Team("Orbits",15,15),
            new Team("Blazers",18,9),
            new Team("Rockets",15,7)
        ));
        System.out.println(board.ranking());
    }
}
```

Expected ranking: Falcons(18,12), Blazers(18,9), Comets(18,9), Orbits(15,15), Rockets(15,7). The name tie-breaker places Blazers before Comets.

5.3 Wildcards, comparator correctness and faulty designs

List<? extends T> is useful for accepting a list of subtype values and adding them to a collection of T. It is a producer of T (PECS: Producer Extends). Comparator<? super T> accepts a comparator that can compare T values or a supertype, making the API flexible.

Do not compare integer scores by subtraction (a.points - b.points). Subtraction can overflow for extreme integer values and may violate comparator ordering. Use Integer.compare(a.points,b.points) or Comparator.comparingInt(). Do not omit tie-breakers when the specification requires a deterministic ranking; otherwise distinct teams can compare as equal.

```
Comparator<Team> safe =
    Comparator.comparingInt((Team t)->t.points).reversed()
      .thenComparing(Comparator.comparingInt(
          (Team t)->t.goalDiff).reversed())
      .thenComparing(t->t.name);


// Deliberately faulty: subtraction may overflow
Comparator<Team> faulty =
    (a,b) -> b.points - a.points;
```

The ranking() method copies items before sorting. Thus the internal insertion order remains unchanged. Sorting n items costs O(n log n) time. The copied list uses O(n) extra space; the sort implementation may use additional temporary storage depending on the JDK.

If Team implements Comparable<Team>, ranking can use natural ordering and Collections.sort(list), but that hard-codes one ordering into the type. If three stable sorts are used, sort by the least significant key first (name ascending), then goal difference descending, then points descending. Each sort costs O(n log n), so three sorts are still O(n log n) asymptotically but with a larger constant factor. A single comparator chain is clearer and usually performs one sort.

Approach	Sort work	Extra space	Reuse
Comparator chain	One O(n log n) sort	O(n) copied list	High; different comparators per type/use
Comparable	One O(n log n) sort	O(n) copied list	Best only for one natural order
Three stable sorts	Three O(n log n) sorts	O(n) temporary/copy space	More error-prone; sort order must be reversed

			carefully

Q6 – Priority, Order and Fairness in a Queue (8 Marks)

6.1 Why PriorityQueue

A normal FIFO Queue serves the earliest arrival regardless of severity, so a minor case could be treated before a critical case. A PriorityQueue orders patients by severity first (1 is highest priority) and by arrival sequence second. The sequence counter guarantees first-come-first-served among equal severities.

6.2 Completed code

```
import java.util.*;
class Patient {
    final String name; final int severity; final long arrival;
    Patient(String n,int s,long a){name=n;severity=s;arrival=a;}
    public String toString(){return name+"(S"+severity+")";}
}
public class Triage {
    private long counter=0;
    private final PriorityQueue<Patient> queue =
        new PriorityQueue<>(Comparator
          .comparingInt((Patient p)->p.severity)
          .thenComparingLong(p->p.arrival));
    void admit(String name,int severity) {
        if(severity<1 || severity>5)
            throw new IllegalArgumentException("Severity must be 1..5");
        queue.add(new Patient(name,severity,counter++));
    }
    Patient next(){return queue.poll();}
    public static void main(String[] args) {
        Triage t=new Triage();
        t.admit("Asha",3); t.admit("Bala",1);
        t.admit("Charan",2); t.admit("Divya",1);
        t.admit("Ezhil",5); t.admit("Farah",3);
        t.admit("Gokul",2); t.admit("Hari",4);
        Patient p;
        while((p=t.next())!=null) System.out.println(p);
    }
}
```

Expected treatment order:

```
Bala(S1)
Divya(S1)
Charan(S2)
Gokul(S2)
Asha(S3)
Farah(S3)
Hari(S4)
Ezhil(S5)
```

Printing the PriorityQueue object directly does not guarantee sorted iteration. A heap guarantees that its head is the least element; the remaining internal array is only partially ordered. Repeated poll() operations are the correct way to observe treatment order.

6.3 Complexity and growth

Operation	PriorityQueue complexity	Explanation
Admit/add	O(log n)	New element may move up the binary heap
Serve/poll	O(log n)	Root is removed and heap is restored
Peek	O(1)	Root contains highest-priority patient
Space	O(n)	One heap entry per waiting patient

Benchmark with queue sizes 1,000, 10,000 and 100,000. For each size, prebuild the queue, measure a batch of adds and a batch of polls, repeat at least seven times, and record average time per operation. Avoid timing console output as part of the operation.

Queue size	Add batch average	Poll batch average	Average per operation
1,000	Enter measured value	Enter measured value	Calculate
10,000	Enter measured value	Enter measured value	Calculate
100,000	Enter measured value	Enter measured value	Calculate

6.4 Starvation, fairness and deployment

A severity-5 patient may starve if severity-1 to severity-4 arrivals continually enter the queue. Strict priority always selects the most severe available patient, so lower-priority waiting time has no finite upper bound under an unending stream of higher-priority arrivals.

A fair extension is aging: increase a patient's effective priority after each defined waiting interval, while retaining clinical safety rules. For example, after every 30 minutes waiting, promote the patient by one priority level, subject to clinician approval and never above an emergency safety ceiling. Another approach is to reserve capacity for lower-severity cases when safe.

A city hospital receiving 5,000 patients per day averages about 0.058 patients per second across 24 hours (5,000 / 86,400). Even if arrivals are uneven and peak rates are much higher, O(log n) heap insertion/removal is efficient for realistic waiting-queue sizes. The collection alone does not make a hospital-grade system: clinical reassessment, multiple doctors, cancellation, audit logging, persistence, concurrency control and emergency overrides are also required.

6.5 Alternative implementations

Implementation	Admit	Serve next	Space	Assessment
PriorityQueue heap	O(log n)	O(log n)	O(n)	Balanced general-purpose choice
Sorted LinkedList	O(n) to find insertion point	O(1) remove head	O(n)	Fast next removal but costly insertion
Five FIFO queues	O(1) enqueue into severity queue	O(1) check/remove from highest non-empty queue	O(n)	Excellent for exactly five fixed priority levels

Five FIFO queues are attractive when there are only five fixed severities and FIFO order is required within each severity. To serve, inspect queues from severity 1 to 5 and remove from the first non-empty queue. Since there are only five queues, checking the heads is O(1) with a fixed bound.

Q7 – Diagnose, Redesign and Validate (13 Marks)

7.1 Selected limitation

Selected original solution: repeated String concatenation inside the railway ticket-generation loop from Q1. This is a significant limitation because each append can create a new immutable String and copy the accumulated prefix. As ticket field count and ticket volume increase, cumulative character copying and temporary allocation grow rapidly.

A. Diagnose the bottleneck (2 Marks)

The bottleneck is not the final ticket output itself; it is repeated reconstruction of the growing prefix. If a ticket has n similarly sized fields, the copied prefixes are proportional to $1 + 2 + \ldots + n = n(n+1)/2$, giving $O(n^2)$ copying in the simplified model. Temporary strings increase allocation pressure and can trigger more garbage collection.

B. Define the design objective (2 Marks)

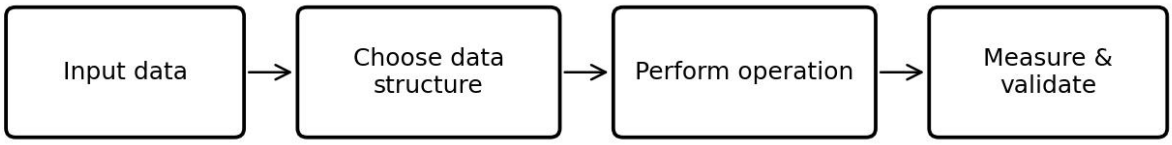
Objective: replace repeated concatenation with one mutable builder per ticket while preserving byte-for-byte ticket content. For the same input, the redesigned program must produce identical output and reduce the character-copying work from quadratic growth in field count to linear growth in total output length. The benchmark should compare average runtime, heap change and estimated temporary objects for 6, 50 and 500 fields.

C. Design the revised solution (4 Marks)

```
static String createTicketOriginal(String[] fields) {
    String result = "";
    for (String field : fields) result += field + "\\n";
    return result;
}
static String createTicketRedesigned(String[] fields) {
    StringBuilder result = new StringBuilder();
    for (String field : fields) result.append(field).append('\\n');
    return result.toString();
}
```

The redesigned method allocates a StringBuilder, appends each field, and converts the buffer to a String once. The builder grows its internal capacity when required; when an approximate final size is known, initialize it with a capacity estimate to reduce resizing. The algorithm preserves field order and delimiters.

D. Flow of the redesigned solution



E. Validation and comparison

Run both implementations using identical arrays of fields. Before timing, compare their returned strings with equals(). Repeat each workload after warm-up and report the average. The table below is intentionally left for actual measurements; do not substitute invented timings.

Field count / ticket count	Original average (ms)	Redesigned average (ms)	Output identical?	Heap/allocations observation
6 / 100	Enter measured value	Enter measured value	Yes/No	Record
50 / 10,000	Enter measured value	Enter measured value	Yes/No	Record
500 / 1,000	Enter measured value	Enter measured value	Yes/No	Record

F. Defend the final design (2 Marks)

The redesign is expected to improve performance for large tickets because it avoids repeated prefix copying. The magnitude depends on field lengths, JDK implementation, JIT optimization and garbage collection. For very small tickets, the difference may be negligible and String concatenation may be easier to read. StringBuilder is the preferred implementation for repeated appends in loops; String remains the correct immutable type for the completed ticket.

Trade-offs: StringBuilder is mutable and must not be shared unsafely between threads. Each ticket should use its own builder. The final String still requires memory, and builder capacity growth can cause occasional buffer expansion. These costs are predictable and generally lower than repeated creation of intermediate strings.

Overall Conclusion

The experiments and complexity analysis show that collection choice must follow workload. StringBuilder is appropriate for repeated ticket construction; ArrayList suits indexed playlists while ArrayDeque suits double-ended queues; HashSet requires a correct equals/hashCode pair; Map selection depends on lookup and report-order requirements; Comparator chains provide reusable and consistent ranking; PriorityQueue handles severity-first triage with FIFO tie-breaking; and the Q7 redesign addresses the allocation bottleneck in repeated String concatenation.

The final submission should include actual benchmark results from the student's computer. The assignment explicitly expects repeated averaged measurements, raw data, source code, screenshots and a progressively committed GitHub repository. The analysis in this report provides the hypotheses and expected asymptotic behaviour, but it does not claim hardware measurements that have not been run.

Appendix – GitHub Structure and Execution Checklist

```
CSA09-<RegisterNumber>-Assignment2/
  README.md
  Task01_StringBenchmark/
    TicketBenchmark.java
    results.csv
    output.txt
  Task02_Playlist/
    PlaylistComparison.java
    results.csv
  Task03_HashContract/
    StudentSetTest.java
    output.txt
  Task04_PharmacyMaps/
    PharmacyInventory.java
    results.csv
  Task05_Leaderboard/
    LeaderboardDemo.java
    output.txt
  Task06_Triage/
    Triage.java
    results.csv
  Task07_Redesign/
    TicketRedesign.java
    comparison.csv
  screenshots/
```

- Replace the GitHub placeholder URL on the cover page with the working repository link.
- Add name, register number, Java version and compile/run commands to README.md.
- Create one folder for each task and include source files, raw CSV measurements and output screenshots.
- Commit progressively: at least one meaningful commit per task.
- Run all Complete the Code programs and preserve their output.
- Fill every benchmark evidence table with repeated measurements from the target machine.
- Check that code compiles and outputs match the expected logical results.
- Be prepared to explain the design decisions, complexity and measurements in the viva.